

Multi-Layer Perceptron

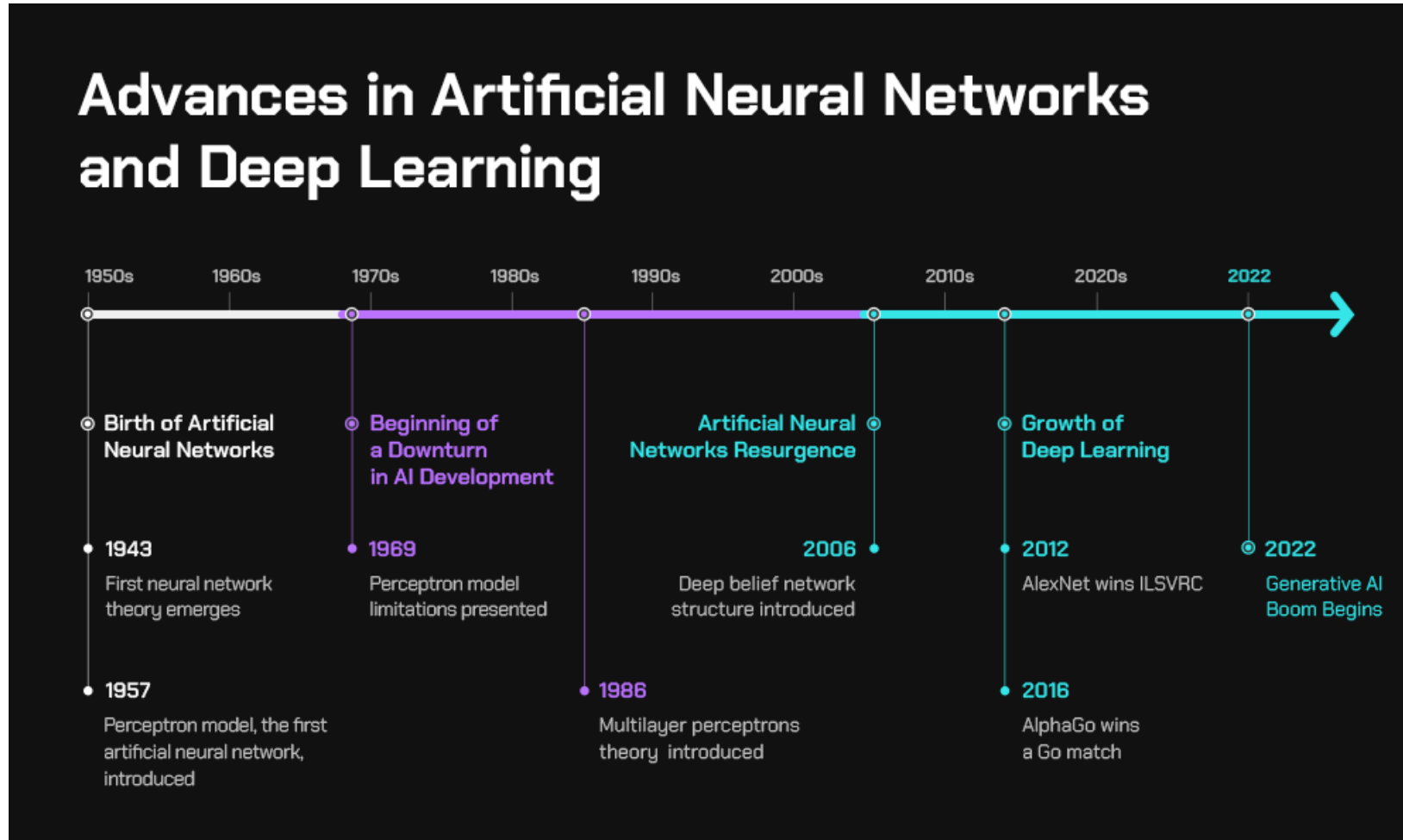
Jihoon Choi

2026.05.16

목차

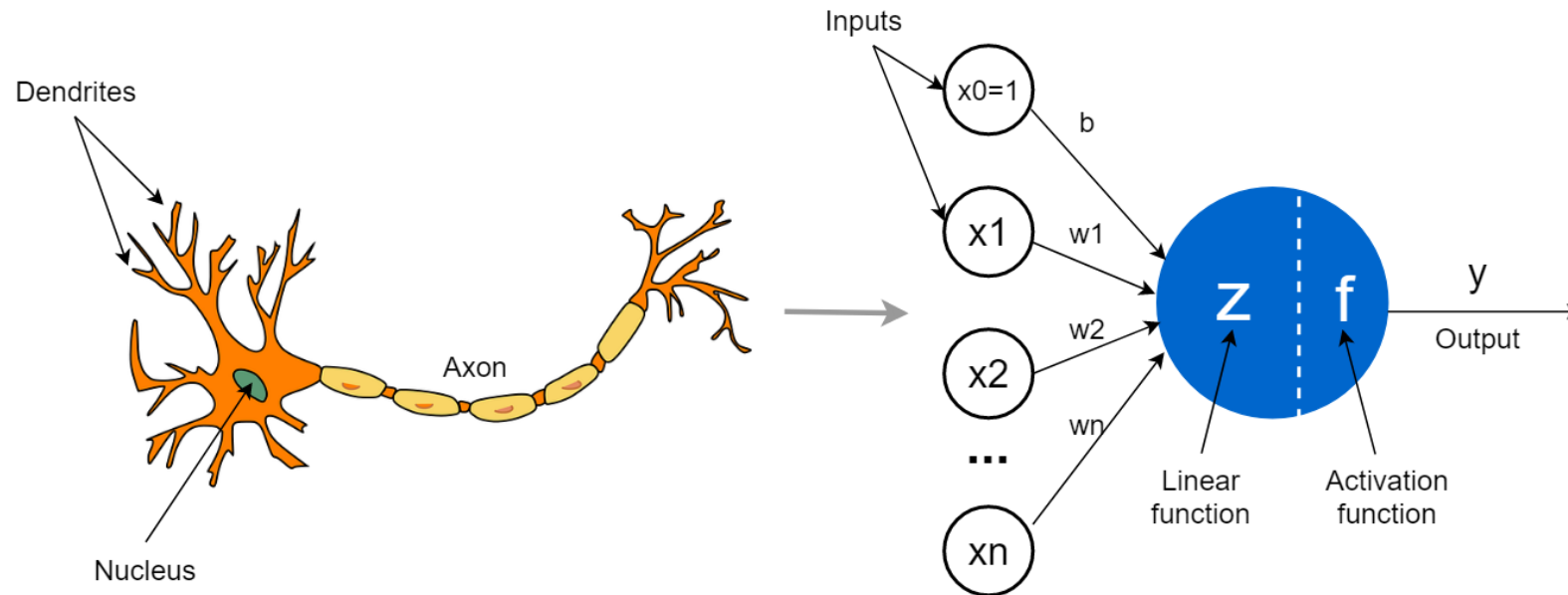
- 인공지능의 발전과 퍼셉트론
- 퍼셉트론의 한계와 해결
- 새로운 학습 방법과 Multi-Layer Perceptron의 등장
- MLP의 한계
- 파이썬 실습

퍼셉트론의 등장



퍼셉트론이란?

- 미국의 심리학자 프랭크 로젠블랫(1928-1971)이 1957년 발표한 개념.



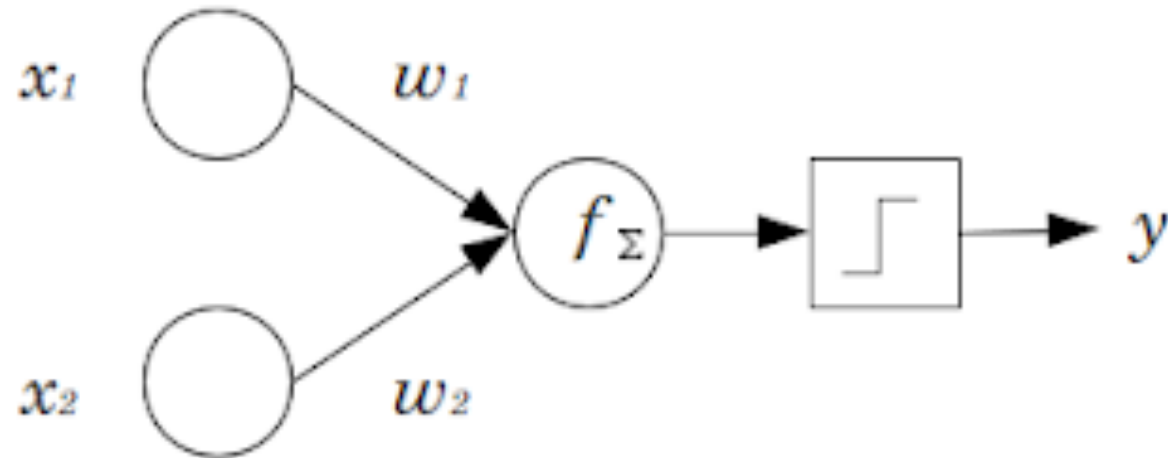
퍼셉트론이란?

- 퍼셉트론의 구조는
 - 입력 신호에 가중치를 곱하고
 - 그 합이 임계값을 넘으면 1
 - 그렇지 않으면 0
- 을 출력하도록 간단하게 제안되었음.

퍼셉트론이란?

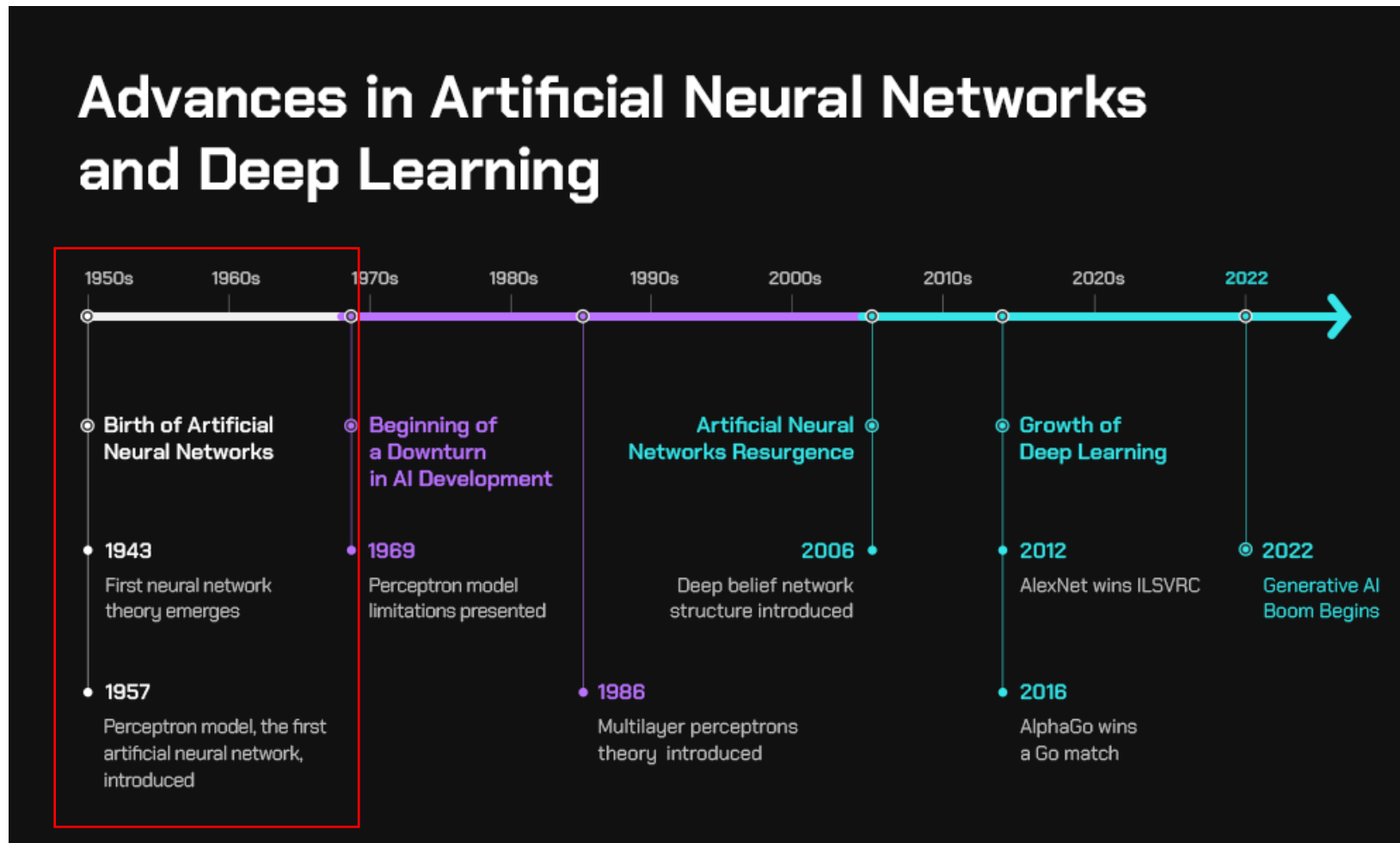
$$x_1 w_1 + x_2 w_2 + b > 0 \Rightarrow y = 1$$

$$x_1 w_1 + x_2 w_2 + b \leq 0 \Rightarrow y = 0$$



퍼셉트론이란?

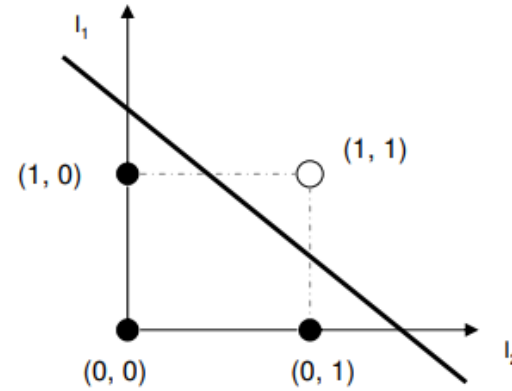
- 왜 퍼셉트론이 이렇게 유명해졌는가?



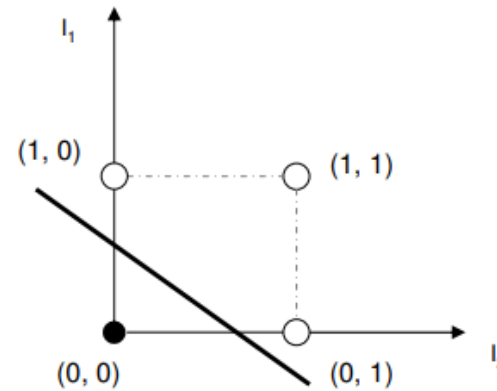
퍼셉트론이란?

- 이러한 선을 어떻게 찾을 수 있을까?

AND		
I_1	I_2	out
0	0	0
0	1	0
1	0	0
1	1	1



OR		
I_1	I_2	out
0	0	0
0	1	1
1	0	1
1	1	1



퍼셉트론이란?

- 퍼셉트론 알고리즘

Algorithm 1: Perceptron Learning Algorithm

Input: Training examples $\{\mathbf{x}_i, y_i\}_{i=1}^m$.

Initialize $\mathbf{w}$ and b randomly.

while *not converged* **do**

Loop through the examples.

for $j = 1, m$ **do**

Compare the true label and the prediction.

$error = y_j - \sigma(\mathbf{w}^T \mathbf{x}_j + b)$

If the model wrongly predicts the class, we update the weights and bias.

if $error \neq 0$ **then**

Update the weights.

$\mathbf{w} = \mathbf{w} + error \times \mathbf{x}_j$

Update the bias.

$b = b + error$

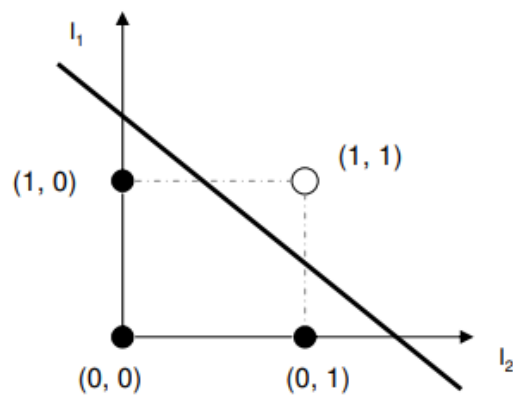
 Test for convergence

Output: Set of weights $\mathbf{w}$ and bias b for the perceptron.

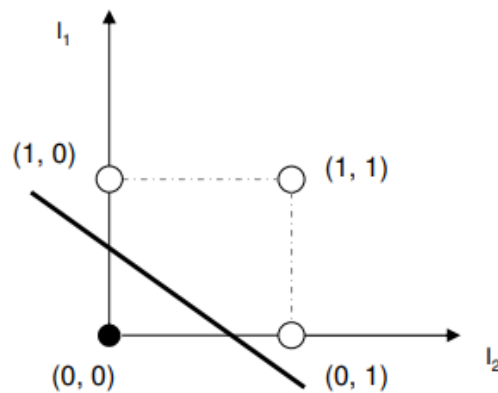
퍼셉트론의 한계

- 마빈민스키가 제안한 XOR 문제

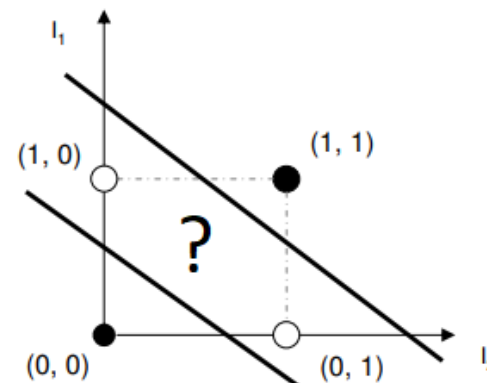
AND		
I_1	I_2	out
0	0	0
0	1	0
1	0	0
1	1	1



OR		
I_1	I_2	out
0	0	0
0	1	1
1	0	1
1	1	1

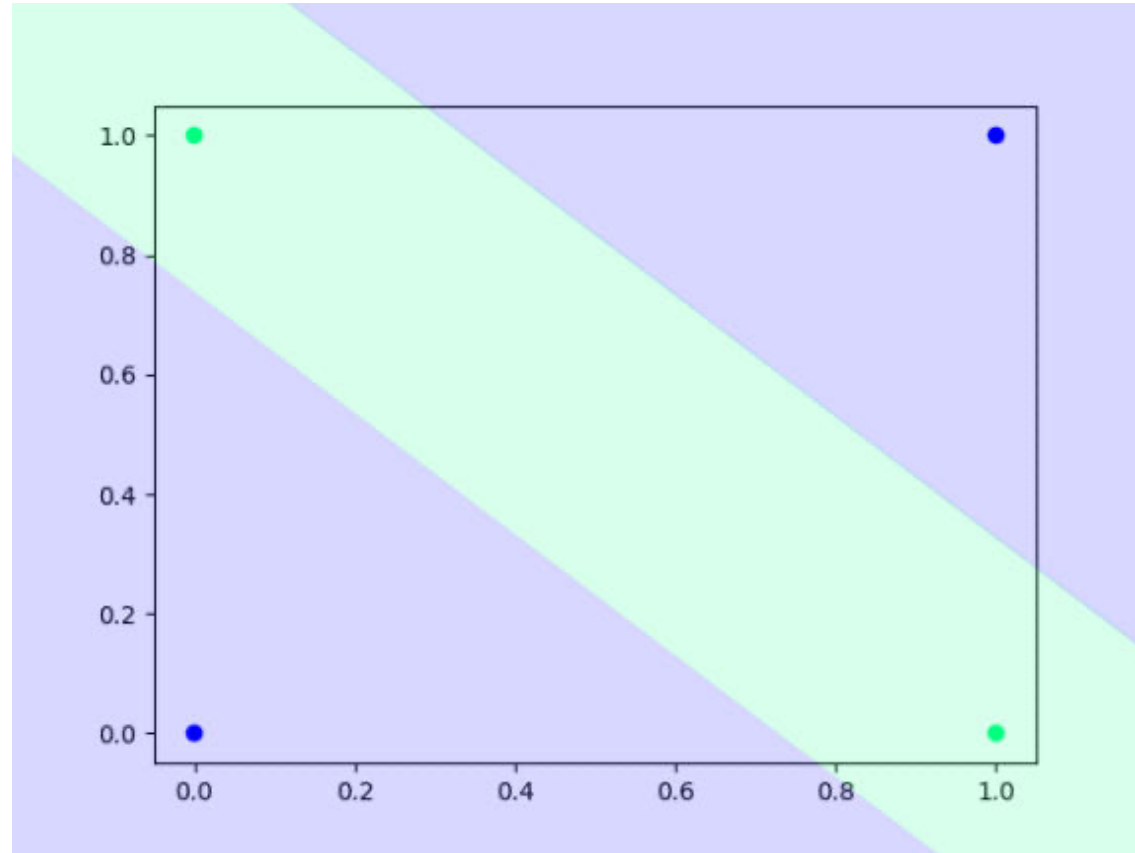


XOR		
I_1	I_2	out
0	0	0
0	1	1
1	0	1
1	1	0



퍼셉트론은 이 문제를 해결할 수 없는가?

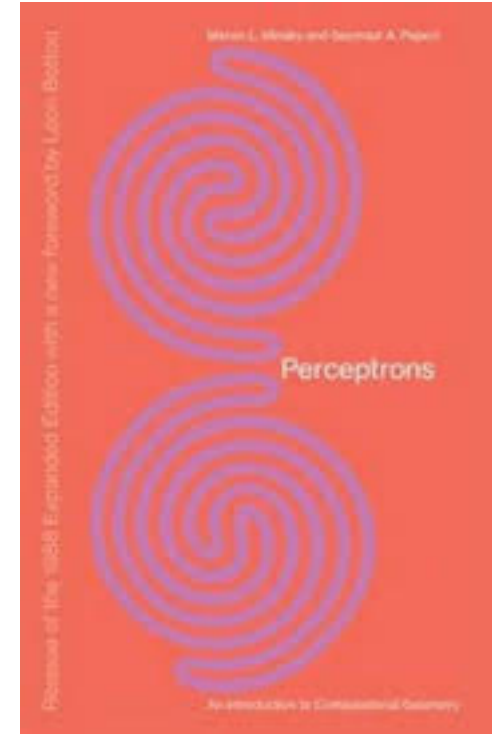
- 2-layer perceptron을 사용하면 이를 해결할 수 있다.



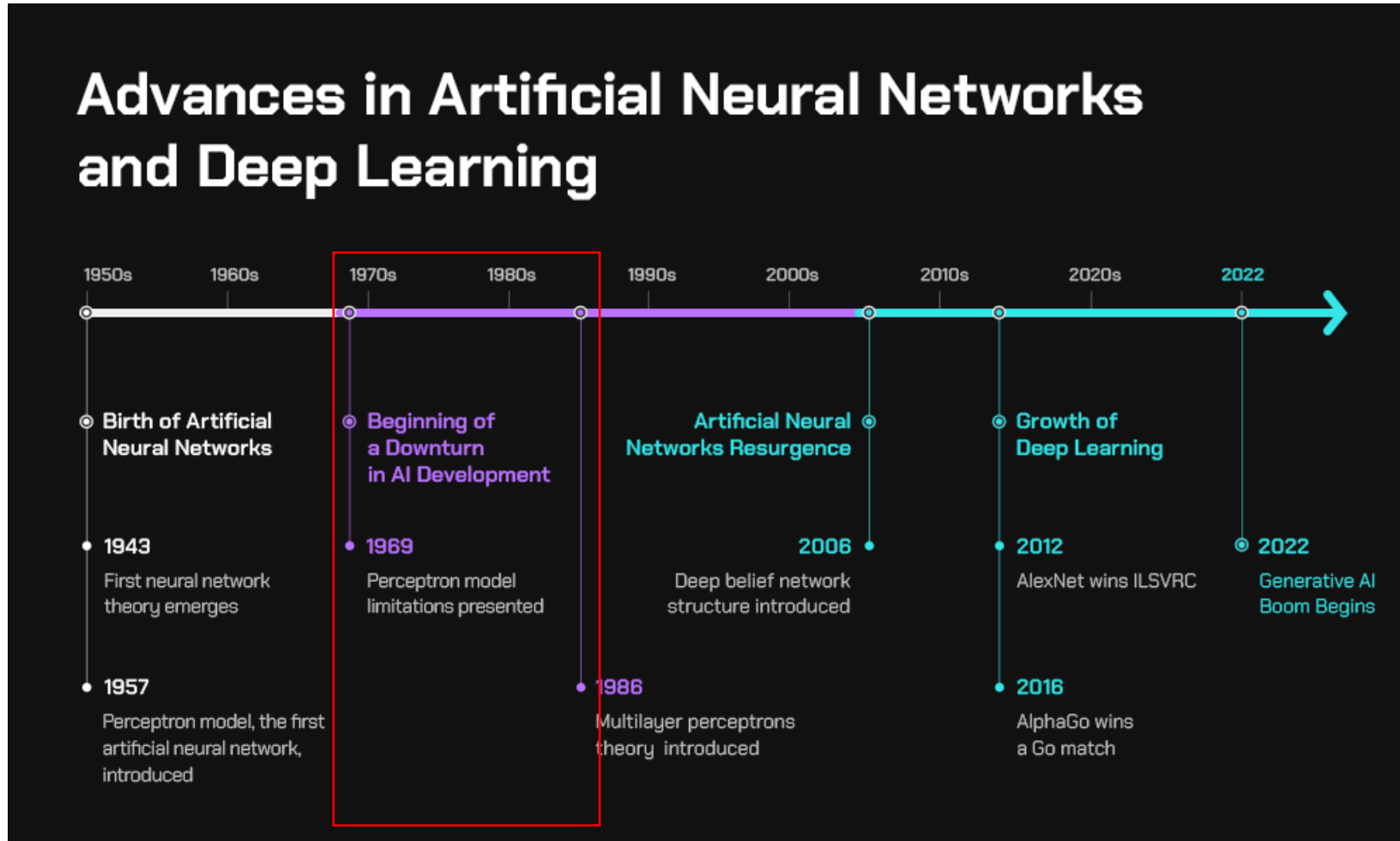
2. 퍼셉트론의 한계와 해결

퍼셉트론은 이 문제를 해결할 수 없는가?

- 이렇게 쌓은 Multi-Layer Perceptron을 학습할 방법이 마땅치 않았다!

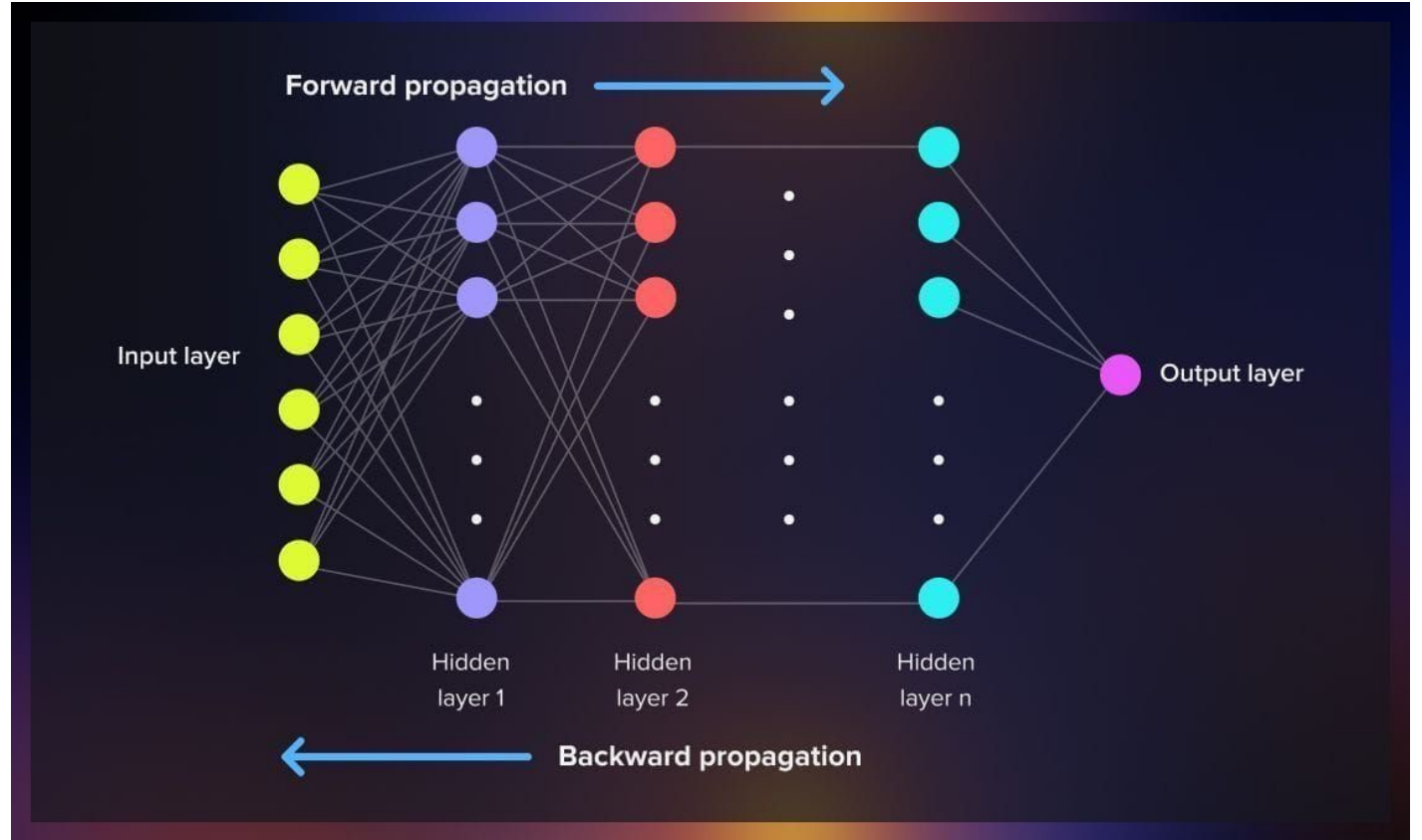


새로운 학습 방법

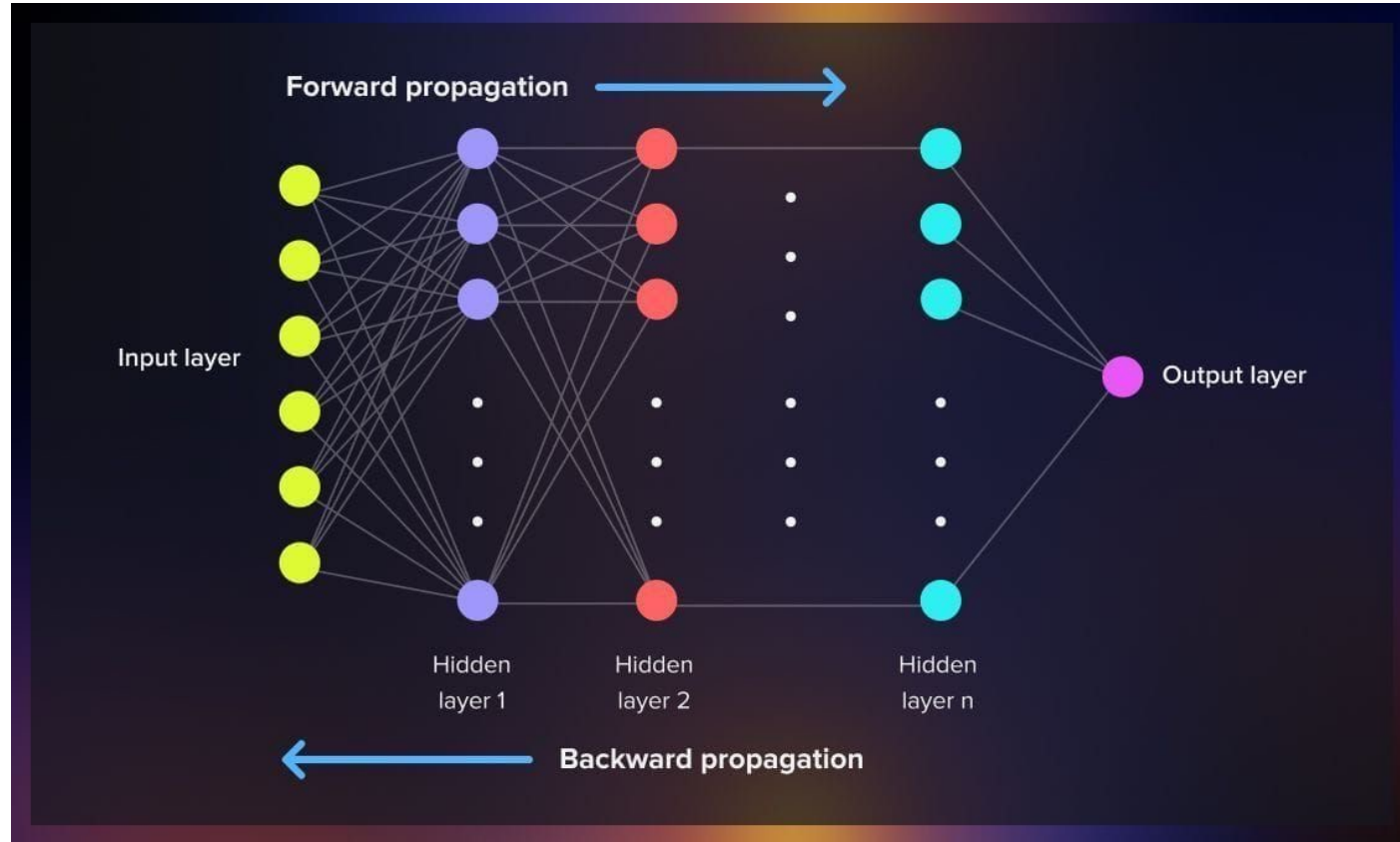


새로운 학습 방법

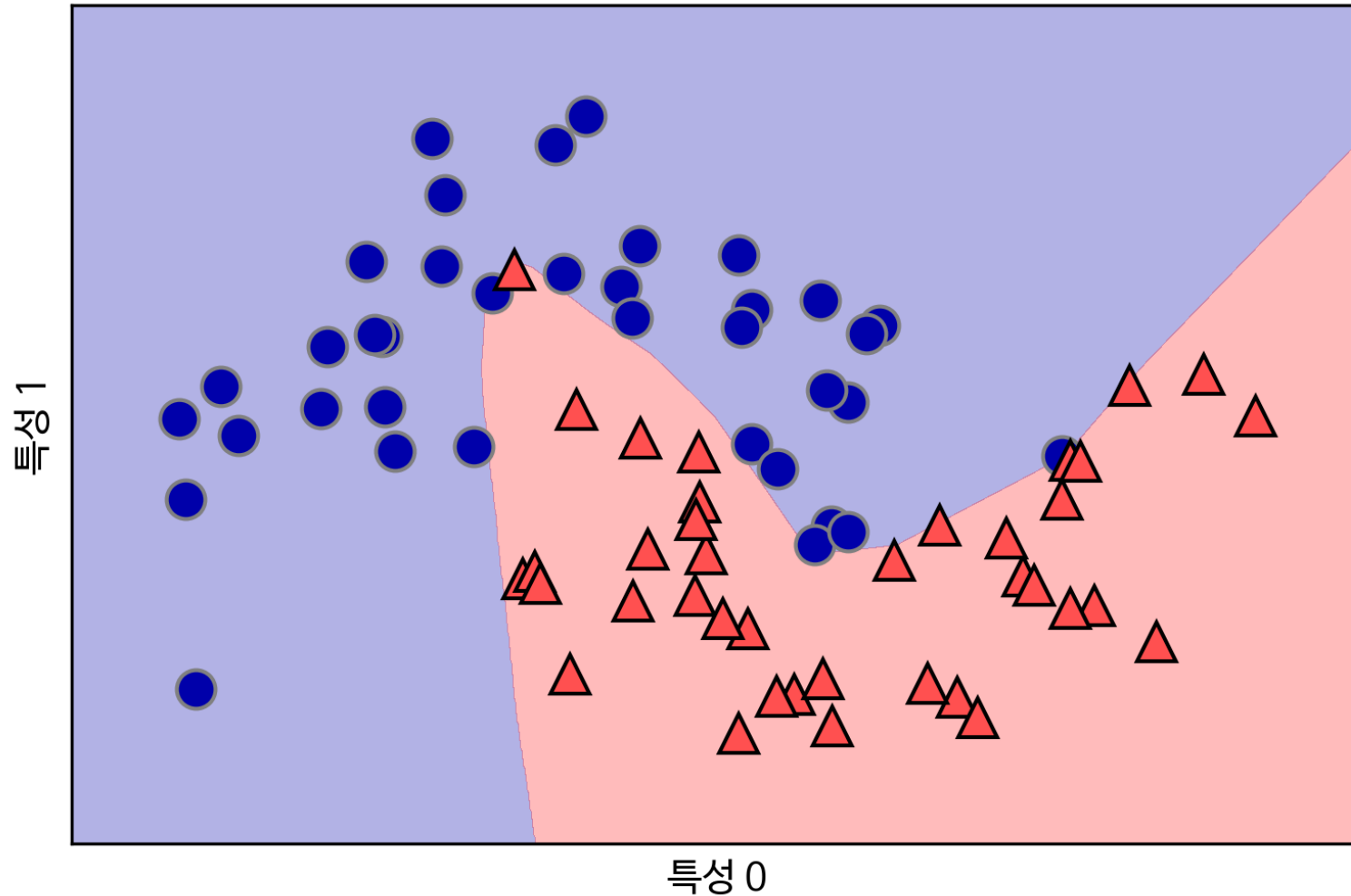
- Backpropagation을 MLP학습에 적용



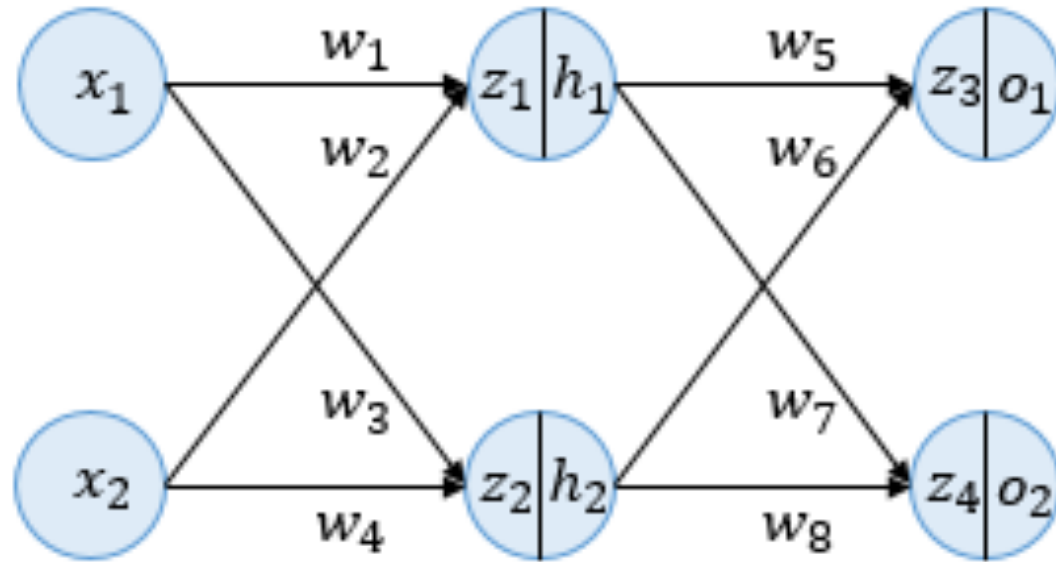
Multi-Layer Perceptron



Multi-Layer Perceptron

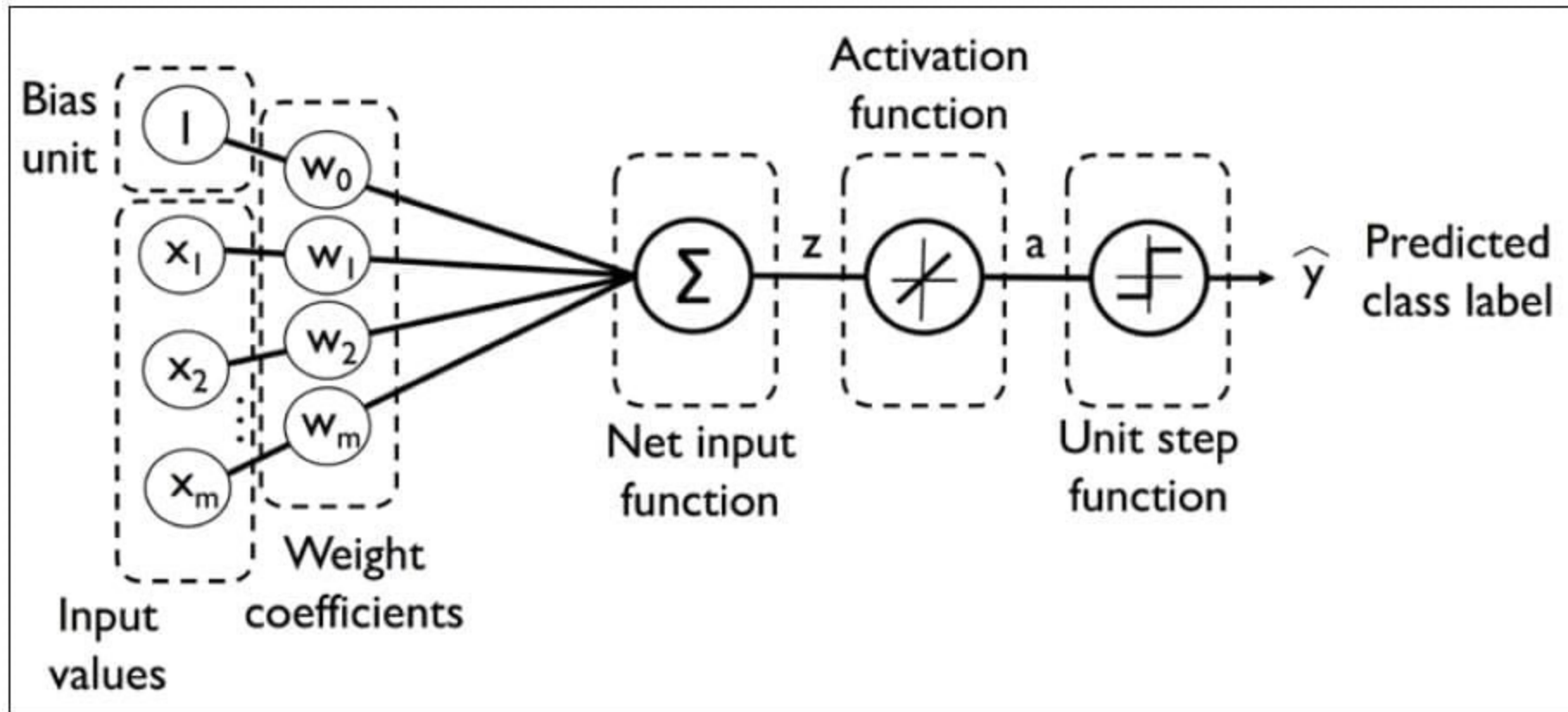


Multi-Layer Perceptron(Forward)



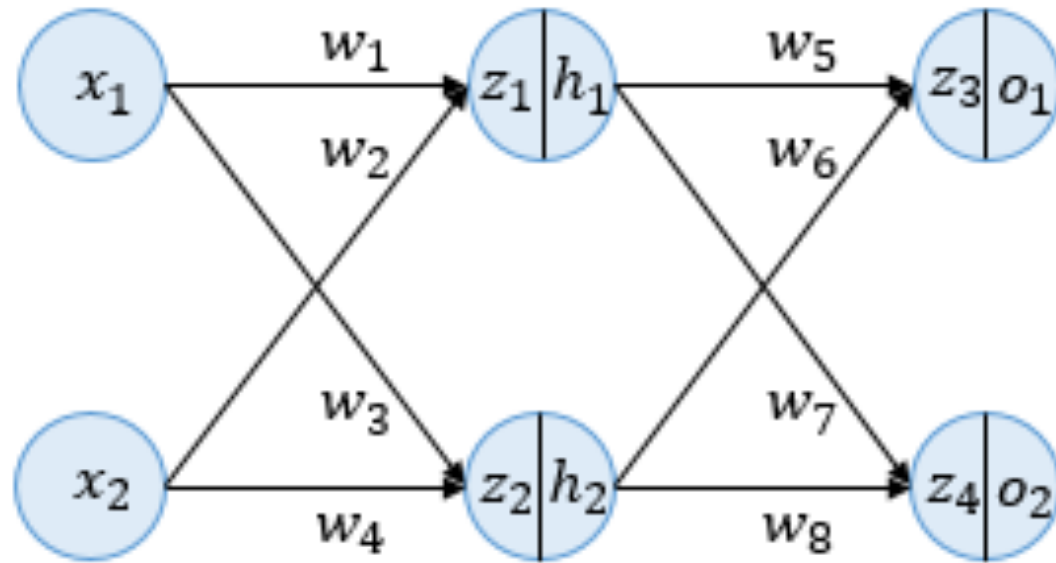
Multi-Layer Perceptron(Activation Function)

- 왜 비선형인가?



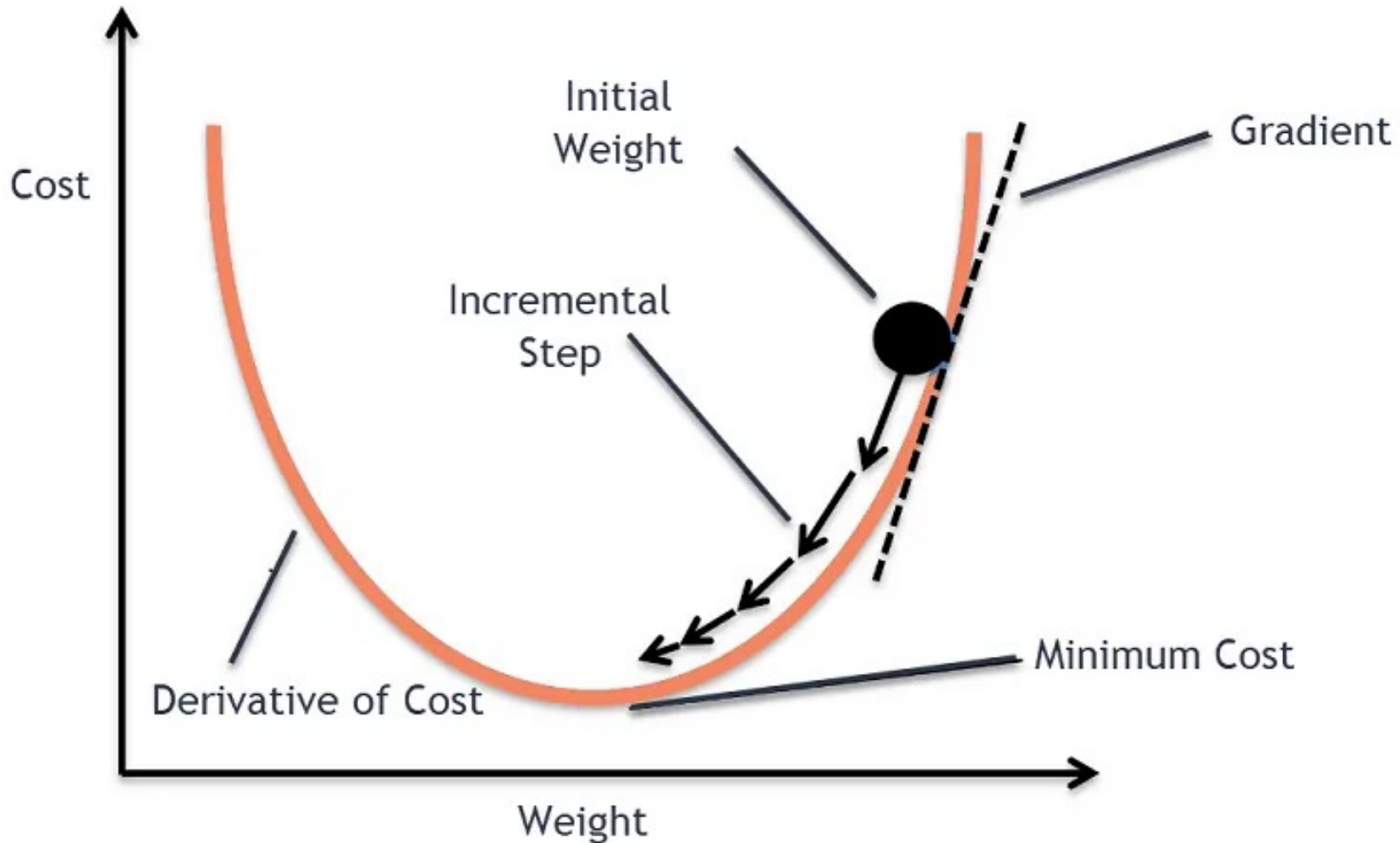
Multi-Layer Perceptron(Backward Propagation)

- 목표: Loss function 최소화

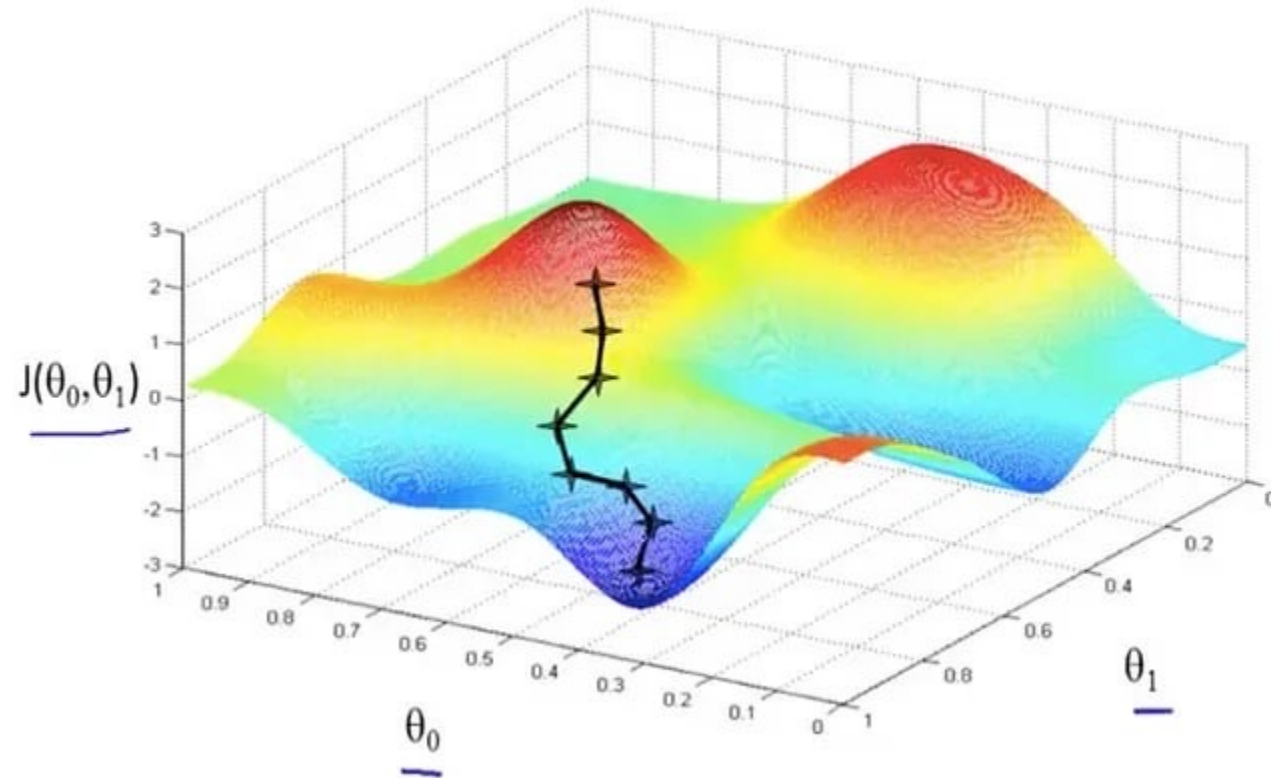


MLP의 학습(Gradient Descent 소개)

- 눈을 가리고 산을 내려오는 것(어느 방향으로, 얼마만큼 갈것인가?)

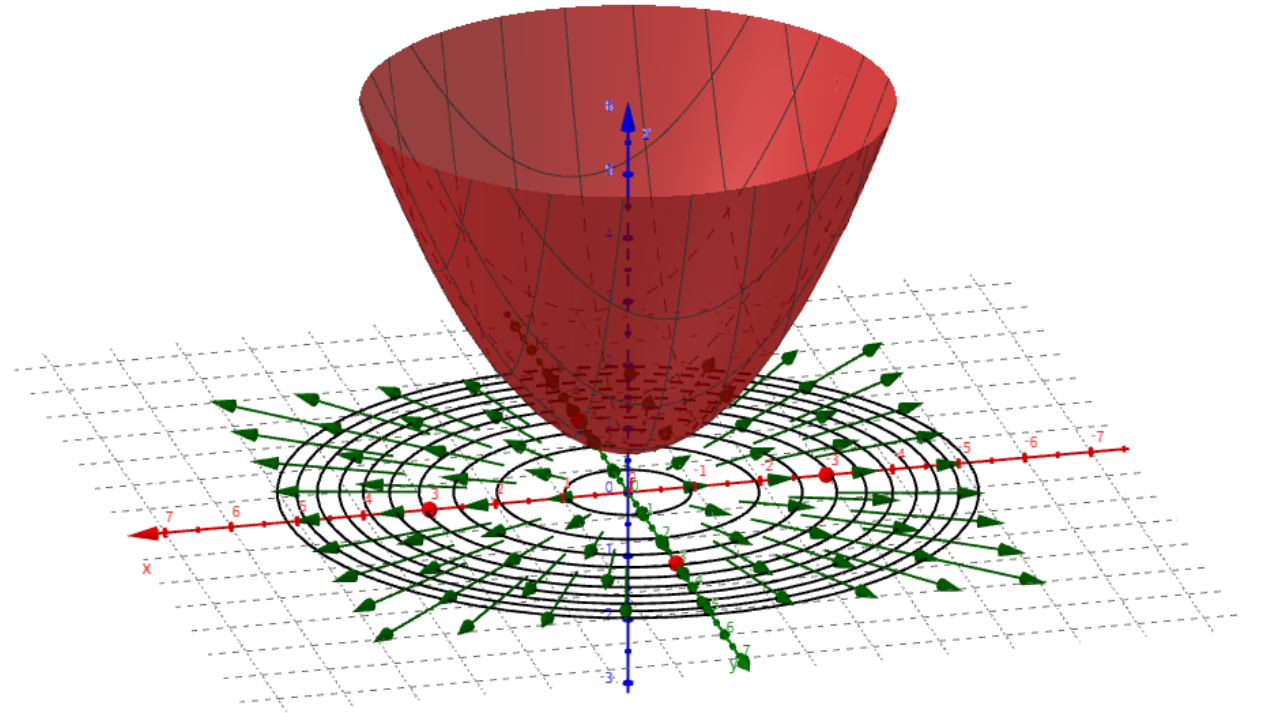
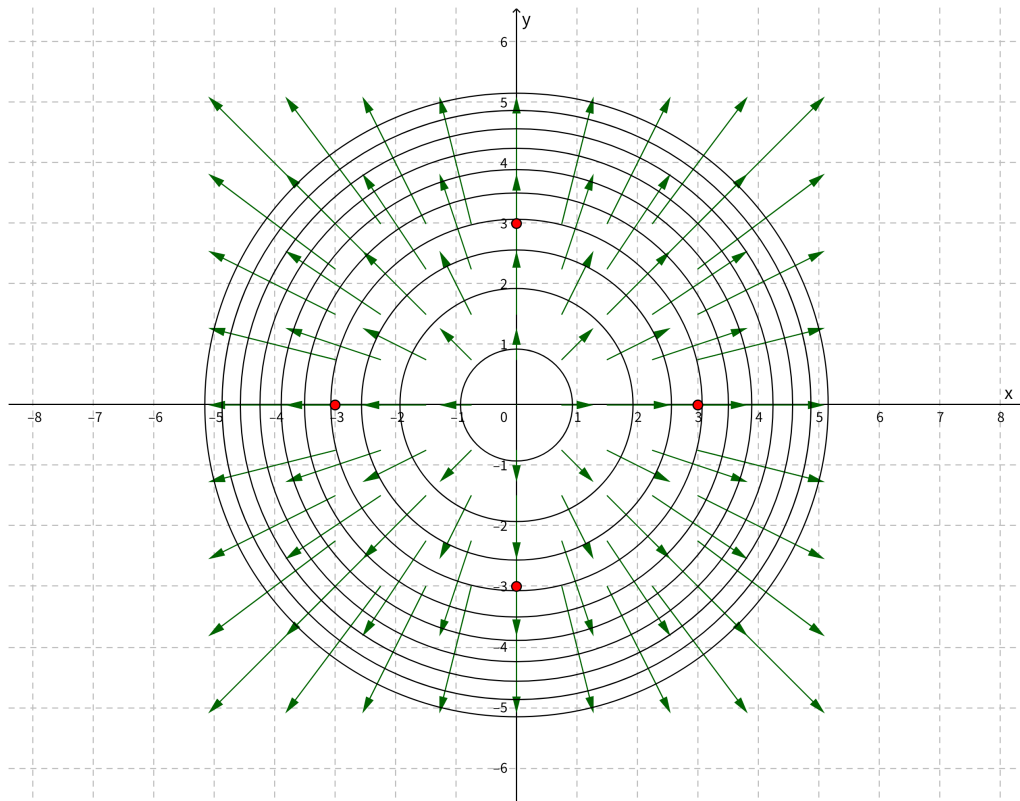


MLP의 학습(Gradient Descent 소개)



MLP의 학습(Gradient Descent 소개)

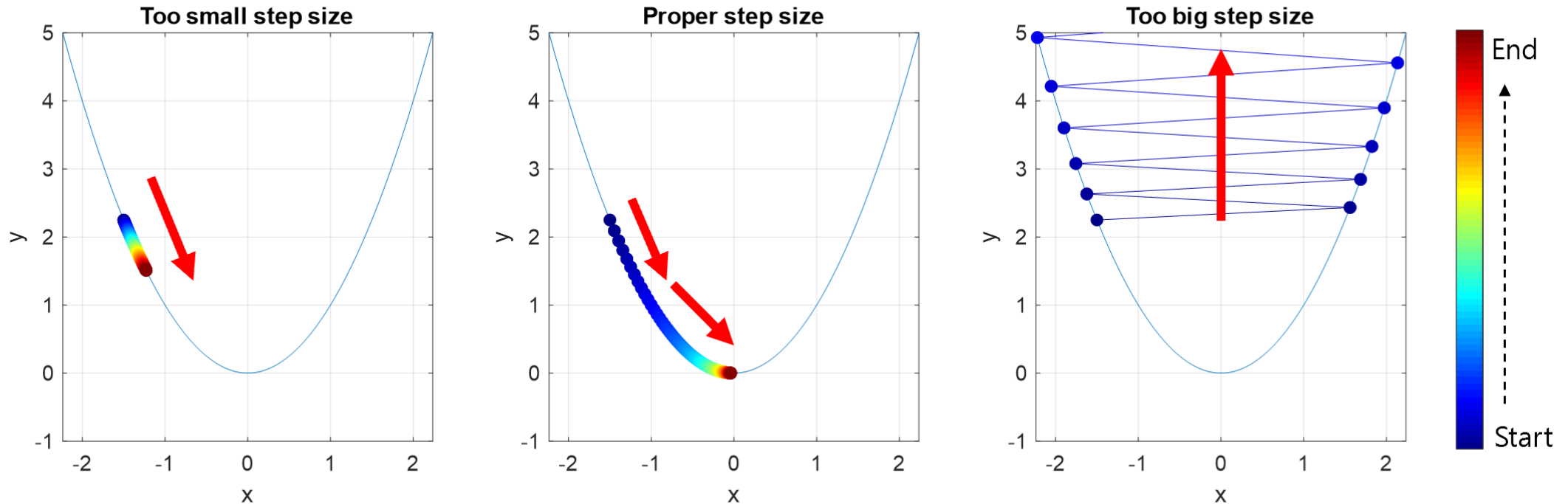
- Gradient의 의미



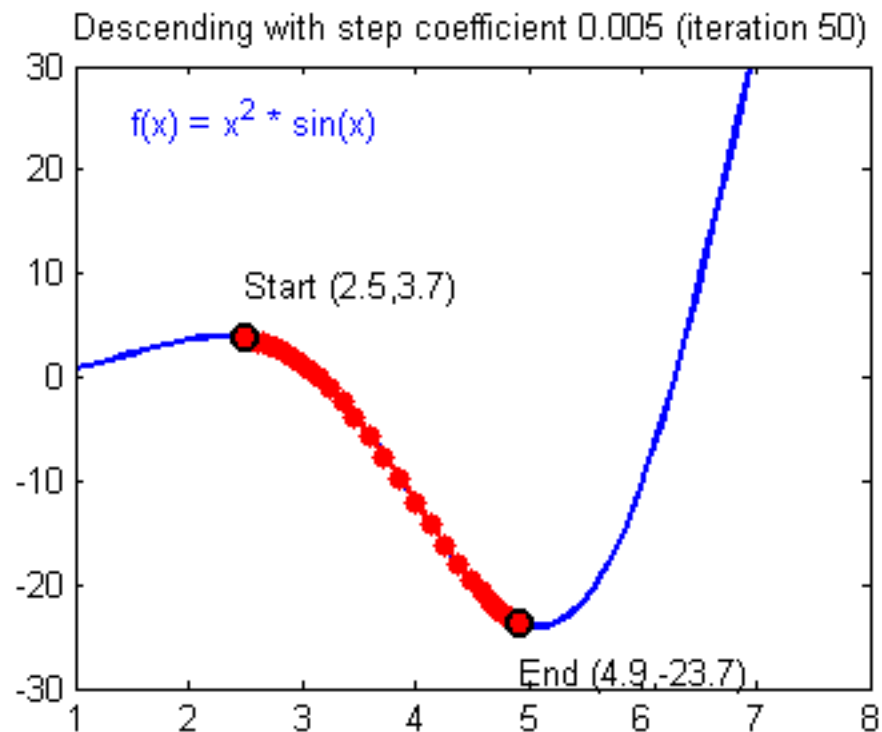
<https://www.geogebra.org/classic/zkdfapfk>

MLP의 학습(Gradient Descent 소개)

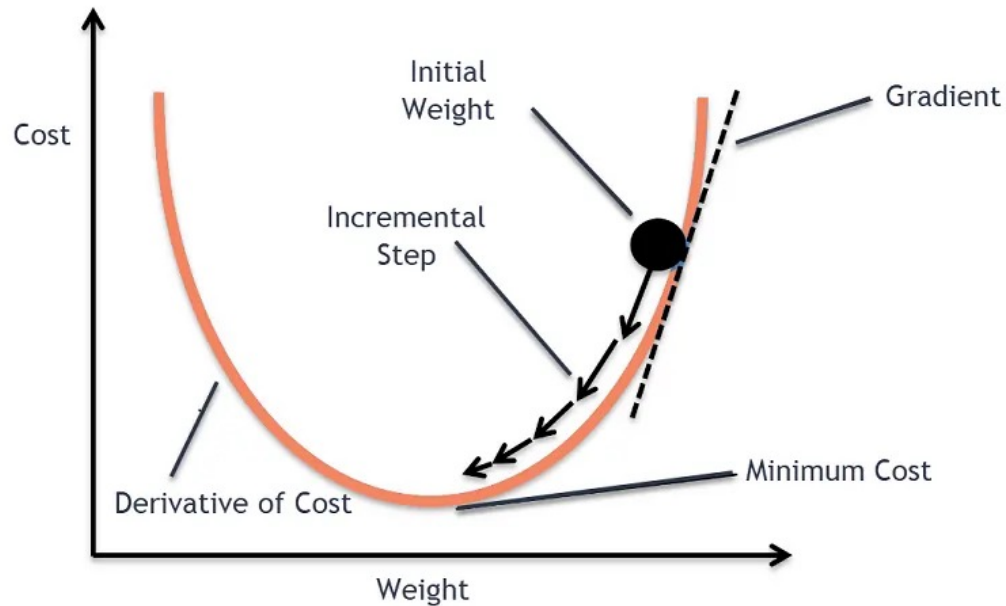
- 눈을 가리고 산을 내려오는 것(어느 방향으로, 얼마만큼 갈것인가?)



MLP의 학습(Gradient Descent 소개)



MLP의 학습(Gradient Descent 소개)



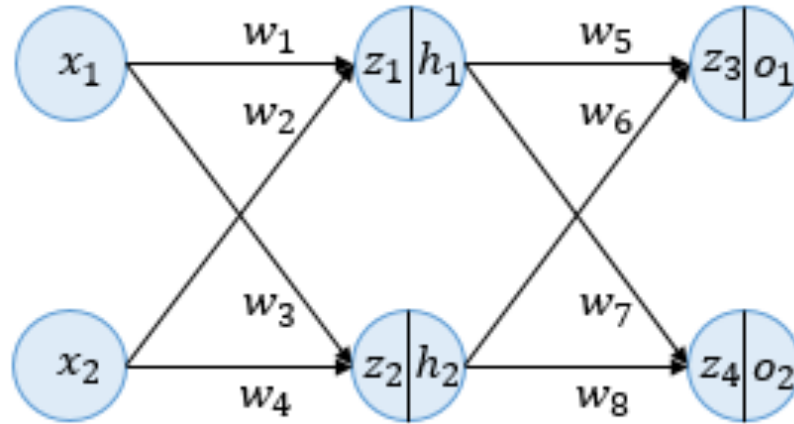
얼마나 움직일지

어디로 움직일지

$$w_1 = w_1 - \alpha \frac{\partial Loss}{\partial w_1}$$

다른 변수를 고정하고 w_1 이 조금 변했을 때,
Loss가 얼마나 변하는가?

MLP의 학습(Back Propagation소개)



$$E_{total} = \frac{1}{2} \left(target_{o_1} - \text{sigmoid}(w_5 h_1 + w_6 h_2) \right)^2$$

$$E_{total} = E_{o_1} + E_{o_2}$$

$$E_{o_1} = \frac{1}{2} (target_{o_1} - o_1)^2$$

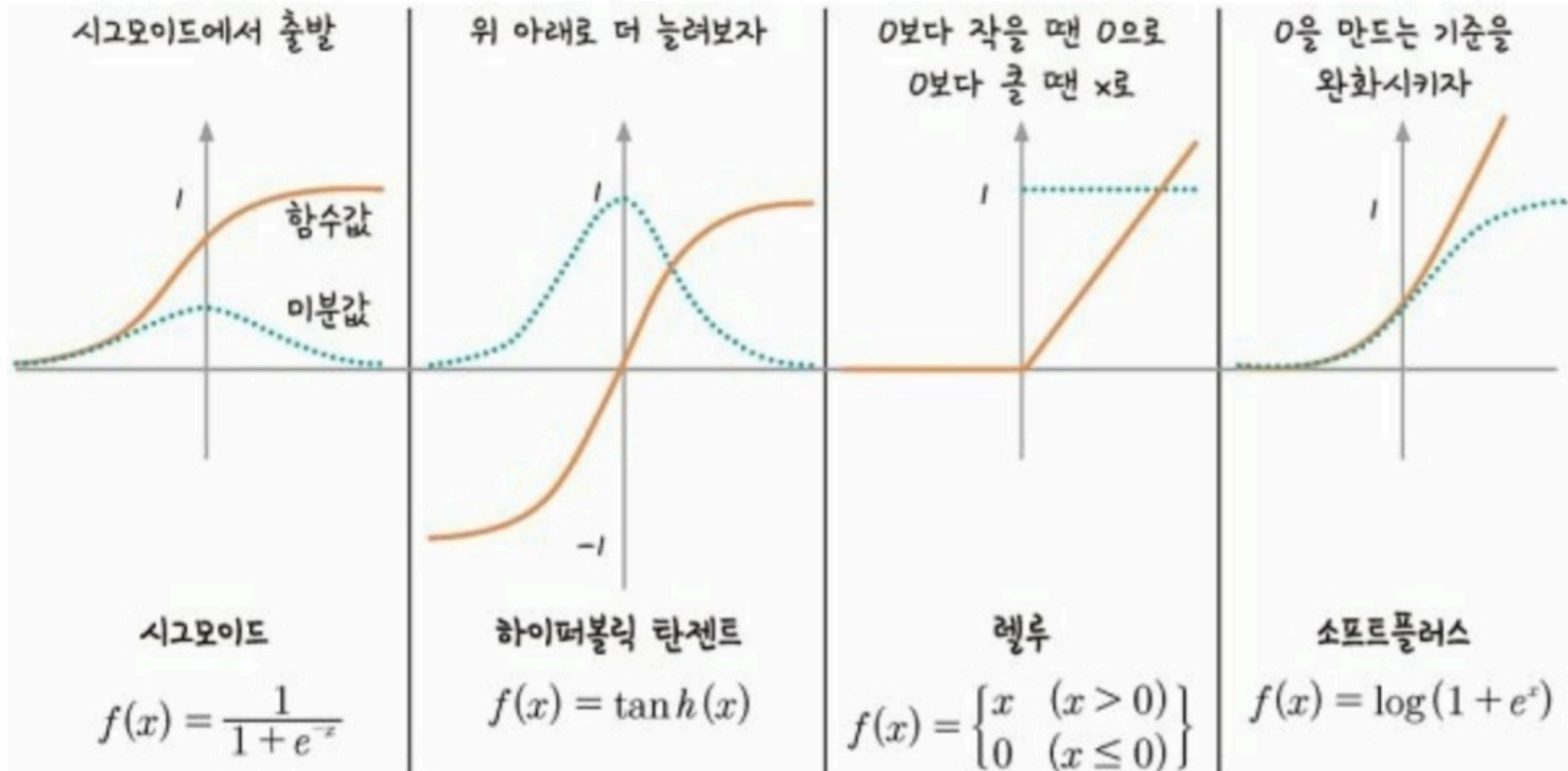
$$o_1 = \text{sigmoid}(z_3)$$

$$z_3 = w_5 h_1 + w_6 h_2$$

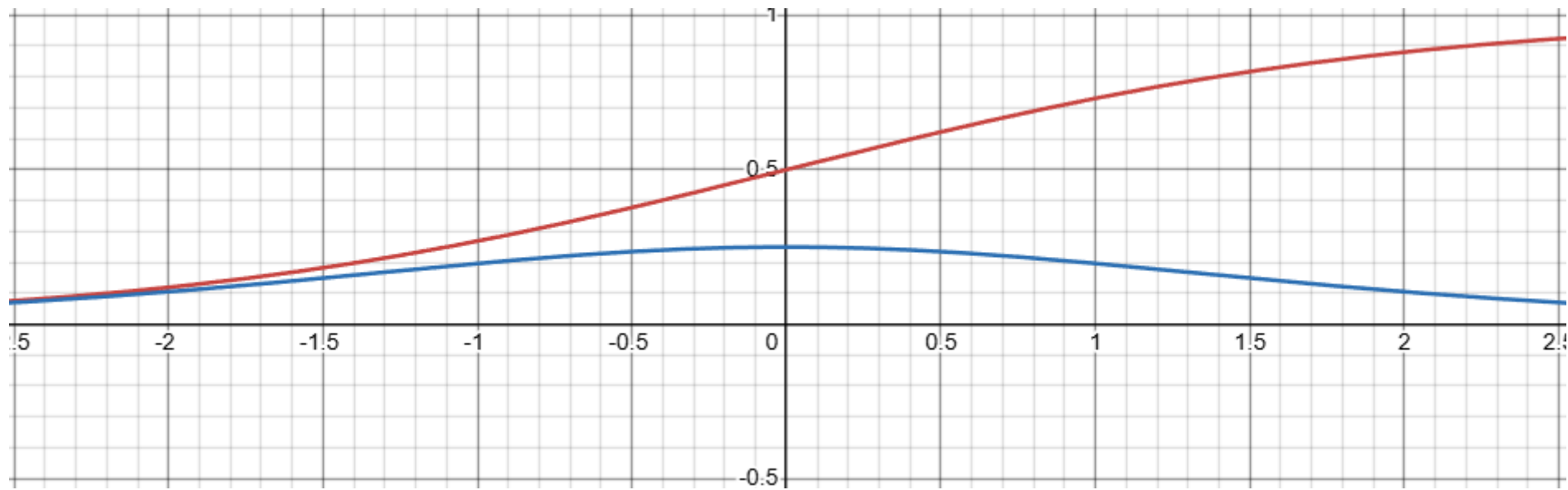


$$\frac{\partial E_{total}}{\partial w_5} = \frac{\partial E_{total}}{\partial o_1} \times \frac{\partial o_1}{\partial z_3} \times \frac{\partial z_3}{\partial w_5}$$

다양한 활성화함수의 등장(Vanishing Gradient)



Sigmoid



$\tanh(x)$

